

Unit:5

WEB AND DATABASE PROGRAMMING WITH .NET [3Hrs]**5.1 Active Server Pages (ASP):**

- Active Server Pages (ASP) is a server-side scripting technology that can be used to create dynamic and interactive Web applications.
- An ASP page is an HTML page that contains server-side scripts that are processed by the Web server before being sent to the user's browser.
- We can combine ASP with Extensible Markup Language (XML), Component Object Model (COM), and Hypertext Markup Language (HTML) to create powerful interactive Web sites.

A) ASP.NET Page Life Cycle

The page life cycle phases are explained below:

Phases	Description
Page request	The page request occurs before the page life cycle begins. When the page is requested by a user, ASP.NET determines whether the page needs to be parsed and compiled (therefore beginning the life of a page), or whether a cached version of the page can be sent in response without running the page.
Start	In the start stage, page properties such as Request and Response are set. At this stage, the page also determines whether the request is a postback or a new request and sets the IsPostBack property.
Initialization	During page initialization, controls on the page are available and each control's UniqueID property is set. A master page and themes are also applied to the page if applicable. If the current request is a postback, the postback data has not yet been loaded and control property values have not been restored to the values from view state.
Load	During load, if the current request is a postback, control property are loaded with information recovered from view state and control state.
Postback event handling	If the request is a postback, control event handlers are called. After that, the Validate method of all validator controls is called, which sets the IsValid property of individual validator controls and of the page.
Rendering	Before rendering, view state is saved for the page and all controls. During the rendering stage, the page calls the Render method for each control, providing a text writer that writes its output to the OutputStream object of the page's Response property.
Unload	The Unload event is raised after the page has been fully rendered, sent to the client, and is ready to be discarded. At this point, page properties such as Response and Request are unloaded and clean-up is performed.

5.2 ASP.Net and Database Application

- ASP.NET is a web development platform, which provides a programming model, a comprehensive software infrastructure and various services required to build up robust web applications for PC, as well as mobile devices.
- ASP.NET is a part of Microsoft .Net platform. ASP.NET applications are compiled codes, written using the extensible and reusable components or objects present in .Net framework.
- These codes can use the entire hierarchy of classes in .Net framework.
- ASP.NET is used to produce interactive, data-driven web applications over the internet.
- It consists of a large number of controls such as text boxes, buttons, and labels for assembling, configuring, and manipulating code to create HTML pages.

The ASP.NET application codes can be written in any of the following languages:

C#

Visual Basic.Net

Jscript

J#

Seven Key Pillars of ASP.Net:

1. **ASP.NET Is Integrated with the .NET Framework:**
 - The .NET Framework is divided into an almost painstaking collection of functional parts, with tens of thousands of types (the .NET term for classes, structures, interfaces, and other core programming ingredients).
 - The massive collection of functionalities that the .NET Framework provides is organized in a way those traditional Windows programmers will see as a happy improvement.
 - Each one of the thousands of classes in the .NET Framework is grouped into a logical, hierarchical container called a namespace.
 - There is a huge class library.
2. **ASP.NET Is Compiled, Not Interpreted:**
 - ASP.NET applications, like all .NET applications, are always compiled.
 - NET applications actually go through two stages of compilation. In the first stage, the C# code is compiled into a compiled into virtually identical IL code.
 - This IL code is an assembly.
3. **ASP.NET Is Multilanguage:**
 - Many programmers 'll probably opt to use one language over another when one develops an application, that choice won't determine what can accomplish with web applications.
 - That's because no matter what language we use, the code is compiled into IL.
4. **ASP.NET Is Hosted by the Common Language Runtime:**
 - The most important aspect of the ASP.NET engine is that it runs inside the runtime environment of the CLR because of benefits like Automatic memory management and garbage collection, type safety, Extensible metadata, structured error handling, Multithreading etc.
 - The whole of the .NET Framework—that is, all namespaces, applications, and classes—is referred to as managed code.

5. ASP.NET Is Object-Oriented:

- ASP provides a relatively feeble object model.
- Not only does our code have full access to all objects in the .NET Framework, but we can also exploit all the conventions of an OOP (object-oriented programming) environment.
- For example, we can create reusable classes, standardize code with interfaces, extend existing classes with inheritance, and bundle useful functionality in a distributable, compiled component.

6. ASP.NET Supports all Browsers:

- Different browsers, versions, and configurations differ in their support of XHTML, CSS, and JavaScript.
- Web developers need to choose whether they should render their content according to the lowest common denominator, and whether they should add ugly hacks to deal with well-known quirks on popular browsers.
- Although we can retrieve information about the client browser and its capabilities in an ASP.NET page, ASP.NET actually encourages developers to ignore these considerations and use a rich suite of web server controls.

7. ASP.NET Is Easy to Deploy and Configure:

- One of the biggest headaches a web developer faces during a development cycle is deploying a completed application to a production server.
- ASP.NET simplifies this process considerably.
- Every installation of the .NET Framework provides the same core classes. As a result, deploying an ASP.NET application is relatively simple.

Server Control:**INTRODUCING SERVER CONTROL:**

- ASP.NET introduces a remarkable new model for creating web pages.
- In old-style web development, programmers had to master the quirks and details of HTML before they could design a dynamic web page.
- ASP.NET solves this problem with a higher-level model of server controls.
- These controls are created and configured as objects.
- *They run on the web server and they automatically provide their own HTML output.* Even better, server controls behave like their Windows counterparts by *maintaining state and raising events* that we can react to in code.

These two different types of controls play subtly different roles:

1) HTML server controls:

- These are server-based equivalents for standard HTML elements.
- They are also useful when migrating ordinary HTML pages or ASP pages to ASP.NET, because they require the fewest changes.

HTML controls have three main features:

- i) ***They generate their own interface:*** We set properties in code, and the underlying HTML tag is created automatically when the page is rendered and sent to the client.

ii) ***They retain their state:*** Because the Web is stateless, ordinary web pages need to do a lot of work to store information between requests. HTML server controls handle this task automatically. For example, if the user selects an item in a list box, that item remains selected the next time the page is generated. Or, if our code changes the text in a button, the new text sticks the next time the page is posted back to the web server.

iii) ***They fire server-side events:*** For example, buttons fire an event when clicked, text boxes fire an event when the text they contain is modified, and so on. Our code can respond to these events, just like ordinary controls in a Windows application. With event-based programming, we can easily respond to individual user actions and create more structured code. If a given event doesn't occur, the event handler won't be executed.

2) Web controls:

- These are similar to the HTML server controls, but they provide a richer object model with a variety of properties for style and formatting details.
- They also provide more events and more closely resemble the controls used for Windows development.
- Web controls also feature some user interface elements that have no direct HTML equivalent, such as the Grandview, Calendar, and validation control.

Following are the features of Web Controls.

i) ***They provide a rich user interface:*** A web control is programmed as an object but doesn't necessarily correspond to a single element in the final HTML page. For example, we might create a single Calendar or Grandview control, which will be rendered as dozens of HTML elements in the final page. When using ASP.NET programs, we don't need to know anything about HTML. The control creates the required HTML tags for us.

ii) ***They provide a consistent object model:*** HTML is full of quirks and idiosyncrasies. For example, a simple text box can appear as one of three elements, including `<textarea>`, `<input type="text">`, and `<input type="password">`. With web controls, these three elements are consolidated as a single *TextBox* control. Depending on the properties we set, the underlying HTML element that ASP.NET renders may differ. Similarly, the names of properties don't follow the HTML attribute names.

iii) ***They tailor their output automatically:*** ASP.NET server controls can detect the type of browser and automatically adjust the HTML code they write to take advantage of features such as support for JavaScript. We don't need to know about the client because ASP.NET handles that layer and automatically uses the best possible set of features. This feature is known as adaptive rendering.

iv) ***They provide high-level features:*** We'll see that web controls allow us to access additional events, properties, and methods that don't correspond directly to typical HTML controls. ASP.NET implements these features by using a combination of tricks.

DATA ACCESS:

- The .NET Framework includes its own data access technology: ADO.NET.
- ADO.NET consists of managed classes that allow .NET applications to connect to data sources (usually relational databases), execute commands, and manage disconnected data.
- The small miracle of ADO.NET is that it allows us to write more or less the same data access code in web applications that we write for client-server desktop applications, or even single-user applications that connect to a local database.

We can access database with or without ado.net also.

Database Access without ADO.NET

- In ASP.NET, there are a few ways to get information out of a database without directly using the ADO.NET classes.
- Depending on needs, we may be able to use one or more of these approaches to supplement our database code (or to avoid writing it altogether).

An option for database access without ADO.NET includes the following:

- **The SqlDataSource control:** The SqlDataSource control allows us to define queries declaratively. We can connect the SqlDataSource to rich controls such as the GridView, and give our pages the ability to edit and update data without requiring any ADO.NET code. Best of all, the SqlDataSource uses ADO.NET behind the scenes, and so it supports any database that has a full ADO.NET provider.

- **LINQ to Entities:** With LINQ to Entities, we generate a data model with the design support in Visual Studio. The appropriate database logic is generated automatically. LINQ to Entities supports *updates, generates secure and well-written SQL statements*, and provides wide-ranging customizability. LINQ to Entities is also the preferred successor to the simpler LINQ to SQL model, which ASP.NET developers have used in the past.

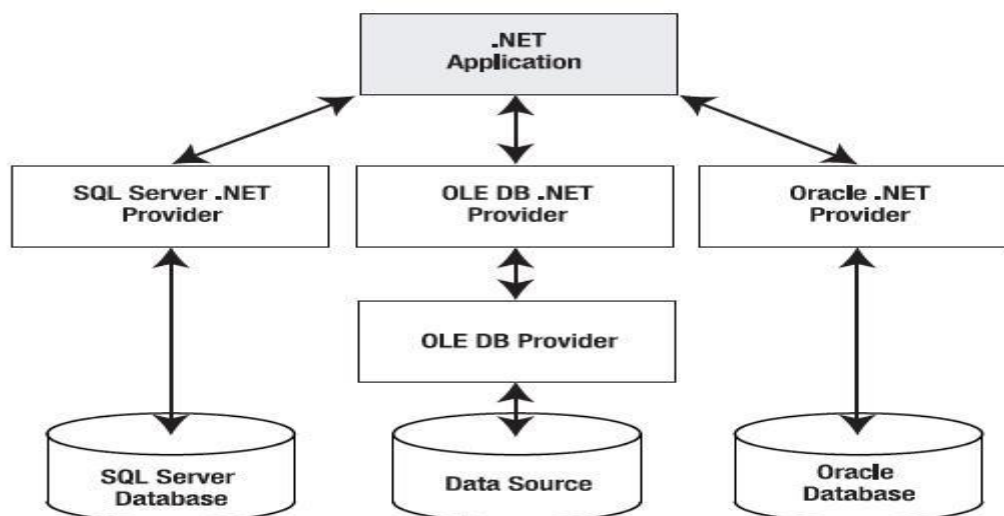
Accessing database with ASP: [imp for exam]

Fig: ADO.Net Architecture

ADO.Net Architecture:

- ADO.NET uses a multilayered architecture that revolves around a few key concepts, such as *Connection*, *Command*, and *DataSet* objects.
- One of the key differences between ADO.NET and some other database technologies is how it deals with the challenge of different data sources.

- ❖ The asp.net framework provides its own data access technology which is known as ADO.Net
- ❖ The ADO.Net provide managed classes that allows the asp.Net applications
 - ✓ To connect to database
 - ✓ Execute sql commands
 - ✓ Retrieve data
- ❖ The ADO.Net uses few key concepts to access the data from database. They are:
 - ✓ Connection
 - ✓ Command
 - ✓ Data reader
 - ✓ Data adapter

The classes that make up a data provider include the following:

- **Connection:** Use this object to establish a connection to a data source.
- **Command:** Use this object to execute SQL commands and stored procedures.
- **DataReader:** This object provides fast read-only, forward-only access to the data retrieved from a query.
- **DataAdapter:** This object performs two tasks. First, *we can use it to fill a DataSet* (a disconnected collection of tables and relationships) *with information extracted from a data source*. Second, *we can use it to apply changes to a data source, according to the modifications we've made in a DataSet*.

ADO.NET Data Providers:

- A data provider is a set of ADO.NET classes that allows us to access a specific database, execute SQL commands, and retrieve data. Essentially, a *data provider is a bridge between our application and a data source*.

➤ **Connection String:**

1) **First Method:**

<configuration>

<connection string>

<add name="db connection string" connection string =Data source= server name;Initial catalog=DatabaseName;userId=sa; password=your password or server"/>

</connection string>

</configuration>

Example:

```
<configuration>
<connection string>
<add name="Mystring" connectionString="data source= dell-pc;Initial
catalog=db_sms;user Id=sa;password=admin"/>
</connevtion string>
</configuration>
```

2) Second Method:

In this method there is no user id and password

```
<configuration>
<connection string>

<add name="db connection string" connection string ="Data source= server
name;Initial catalog=Databasename;Integrated security =SSPI"/>

</connection string>
</configuration>
```

Example:

```
<configuration>
<connection string>

<add name="Mystring" connectionString="data source= dell-pc;Initial
catalog=db_sms;integrated security=SSPI"/>

</connevtion string>
</configuration>
```

Important question for exams:

- 1) Explain the key pillars of ASP
- 2) How do we connect the database using ASP.Net? write the code snippet.
- 3) Define ASP? Explain the Life cycle of ASP page. Explain the server control as well.

Extra Material from This Unit:**ASP.NET Page Life Cycle Events**

Following are the page life cycle events:

PreInit - PreInit is the first event in page life cycle. It checks the IsPostBack property and determines whether the page is a postback. It sets the themes and master pages, creates dynamic controls, and gets and sets profile property values. This event can be handled by overloading the OnPreInit method or creating a Page_PreInit handler.

Init - Init event initializes the control property and the control tree is built. This event can be handled by overloading the OnInit method or creating a Page_Init handler.

InitComplete - InitComplete event allows tracking of view state. All the controls turn on view-state tracking.

LoadViewState - LoadViewState event allows loading view state information into the controls.

LoadPostData - During this phase, the contents of all the input fields are defined with the <form> tag are processed.

PreLoad - PreLoad occurs before the post back data is loaded in the controls. This event can be handled by overloading the OnPreLoad method or creating a Page_PreLoad handler.

Load - The Load event is raised for the page first and then recursively for all child controls. The controls in the control tree are created. This event can be handled by overloading the OnLoad method or creating a Page_Load handler.

LoadComplete - The loading process is completed, control event handlers are run, and page validation takes place. This event can be handled by overloading the OnLoadComplete method or creating a Page_LoadComplete handler

PreRender - The PreRender event occurs just before the output is rendered. By handling this event, pages and controls can perform any updates before the output is rendered.

PreRenderComplete - As the PreRender event is recursively fired for all child controls, this event ensures the completion of the pre-rendering phase.

SaveStateComplete - State of control on the page is saved. Personalization, control state and view state information is saved. The HTML markup is generated. This stage can be handled by overriding the Render method or creating a Page_Render handler.

UnLoad - The UnLoad phase is the last phase of the page life cycle. It raises the UnLoad event for all controls recursively and lastly for the page itself. Final cleanup is done and all resources and references, such as database connections, are freed. This event can be handled by modifying the OnUnload method or creating a Page_UnLoad handler.